

AR-Musician: Playing Instruments the Futuristic Way

Matan Davidi[†]

Cyril Moser[†]

Gent Serifi[†]

Nicola Studer[†]

Ata Çelen[†]

[†]ETH Zurich, Switzerland



Figure 1. **Overview of AR-Musician.** To enable a frictionless setup, we generate a photorealistic synthetic dataset (a) to train a lightweight keypoint detector for automatic piano registration (b). For bongos, we utilize a training-free semantic segmentation approach based on reference features (c). These pipelines run locally on a Meta Quest 3, enabling precise spatial alignment for an immersive, gamified learning experience where notes fall directly onto the physical keys (d) or drums (e).

Abstract

We present AR-Musician, a fully standalone mixed reality system for guided music learning on the Meta Quest 3. It automatically detects and spatially registers physical musical instruments in an Augmented Reality (AR) environment, removing the need for manual calibration. We introduce a synthetic 3D piano dataset and novel detection pipelines for piano and bongo instruments, demonstrating support for multiple instrument types. Our results show that accurate instrument localization and registration are feasible on consumer-grade, standalone Mixed Reality (MR) hardware, paving the way for scalable, instrument-agnostic mixed reality music tutors. Using our implemented multiplayer mode, players can share the fun of learning new instruments. Our code and data are publicly available on [GitHub](#) and [HuggingFace](#).

1. Introduction

Learning a musical instrument is a cognitively demanding task, often hindered by the fragmented interaction between sheet music and the physical instrument. Traditional practice suffers from the split-attention effect [26], where the learner’s gaze must constantly shift between the notes and their hands, disrupting the sensorimotor loop. MR offers a compelling alternative by directly overlaying visual guidance onto real instruments, unifying notation and interaction within a single perceptual space. However, most existing MR systems used for this task rely on manual setup,

external sensors, or server-side inference, which limits accessibility, scalability, and real-time responsiveness.

In this work, we present a fully standalone MR instrument tutor that runs entirely on the Meta Quest 3 headset [11], without any requirement for server connectivity, except for multiplayer, or any cloud-based inference. All computer vision models execute locally on the device, ensuring low latency and robustness for immersive musical practice. In summary, our work proposes the following contributions:

- A new synthetic dataset for 3D piano imagery, enabling precise geometric supervision without costly manual annotation.
- A CNN-based piano keypoint detector trained purely on synthetic data, that enables reliable instrument registration from passthrough images.
- A novel training-free single-shot bongo detection framework that leverages semantic features from a pre-trained image encoder to ensure robust detections.

To the best of our knowledge, this is the first MR system to automatically detect and spatially register musical instruments in an AR environment without manual calibration.

By combining on-device inference, automatic instrument detection, accurate 3D spatial alignment, and out-of-the-box multiplayer, our approach significantly lowers the entry barrier for MR-assisted music learning and establishes a foundation for scalable, instrument-agnostic mixed reality tutors running entirely on consumer-grade standalone hardware.

2. Related Work

Consumer Mixed Reality Applications Leading consumer applications like *PianoVision* [20] and *Magic Keys* [9] successfully gamify learning but typically rely on MIDI connections or manual alignment of virtual guides. Similarly, open-source alternatives like *VRtuos* [16] require users to manually resize and position virtual meshes to match reality. Our work removes these friction points by eliminating the dependency on manual calibration and external hardware through fully automatic, on-device instrument registration.

Academic Augmented Reality Systems Early AR research demonstrated the cognitive benefits of projecting notes onto keys [21] but relied on immobile external projectors. Subsequent headset-based systems, such as *HoloKeys* [10], improved portability but introduced visual clutter by requiring fiducial markers for tracking, while mobile solutions [13] limited the user’s vision with a smartphone placed upon a tripod. We advance this domain by replacing external sensors and markers with markerless, semantic keypoint detection running locally on consumer hardware.

Virtual Reality and Haptics Pure VR rhythm games like *Paradiddle* [27], *Virtuoso* [4], and *Smash Drums* [23] offer engaging simulations but suffer from “air-drumming”, which lacks the passive haptic feedback essential for muscle memory [1]. By anchoring our system in Mixed Reality, we combine the visual interactivity of VR with the crucial tactile feedback of physical instruments.

3. Automatic Instrument Detection

Our systems for both instrument types follow a unified methodology that starts with detecting the minimal set of keypoints required to define the instrument’s geometry. For the piano, this set consists of the two top corners and the bottom-right corner of the white-key area. These points uniquely span a 3D plane, which we then use to align the musical cues with the physical keys. For the bongos, we detect three points on each drum’s circumference to define a circle.

Our detection models leverage the Meta Quest 3’s machine learning capabilities to locate these keypoints within the passthrough feed, which provides images at a resolution of 1280×960 px [18], downsampled to 320×240 px. This data is processed locally on the onboard Snapdragon XR2 Gen2 (4nm) chipset [8, 11]. Once the 2D keypoints are detected (Sec. 3), we utilize the calibrated camera intrinsics to cast virtual rays from the image space into the 3D environment. By leveraging Meta’s Environment Raycasting API [17] and the headset’s integrated depth sensor [11], we determine the intersections of these rays with the real-world

scene. These intersections provide the final 3D keypoint coordinates used for instrument alignment.

3.1. Piano Detection

Whereas recent models like SAM3 [3] or YOLOv9 [30] can detect the piano either on a per-pixel basis or with an axis-aligned bounding box, this information is insufficient or too inaccurate to get the necessary keypoints. For this reason, we decided to design custom detection pipelines tailored to our application.

3.1.1 SynthesiaSet

As there are no publicly available large-scale datasets depicting pianos from egocentric viewpoints, we created our own custom dataset, the *SynthesiaSet*, using a novel synthetic pipeline. To minimize the synth-to-real gap, we need to consider three core requirements for our dataset: perspective diversity, geometric variety, and photorealistic rendering.

Perspective Diversity To ensure the model generalizes to various AR/VR use cases, the dataset must encompass a wide range of viewing conditions. We achieve this by randomly sampling camera parameters, including the distance to the piano, the azimuth and elevation angles, and the field of view (FoV).

Parametric Mesh Generation A primary challenge in this domain is the lack of standardized dimensions across piano manufacturers [22]. A static 3D model fails to generalize to the subtle variations in key geometry and spacing found in the real world. To address this, we developed a novel parametric piano-mesh generator. Unlike rigid models, our generator accounts for the complex spatial relationships within an octave. Specifically, it can vary the individual widths of white keys (*e.g.*, distinguishing the width of ‘D’ from ‘C’ or ‘E’) and the lateral offset of black keys. This allows us to simulate configurations where an accidental (*e.g.*, C#) is not perfectly centered between its natural neighbors but is slightly shifted. The generator also parameterizes the total key count, the starting key, and the thickness and length of the key tops.

The generation process utilizes Trimesh [5] to produce an initial low-poly base mesh. To avoid the artifacts of unnaturally sharp edges, we implement a refinement pipeline: the mesh is first subdivided using loop-subdivision [15], then smoothed via a Humphrey filter [29] to bevel the corners, and finally processed through quadric mesh decimation [7] to reduce complexity while preserving realistic, rounded geometry.

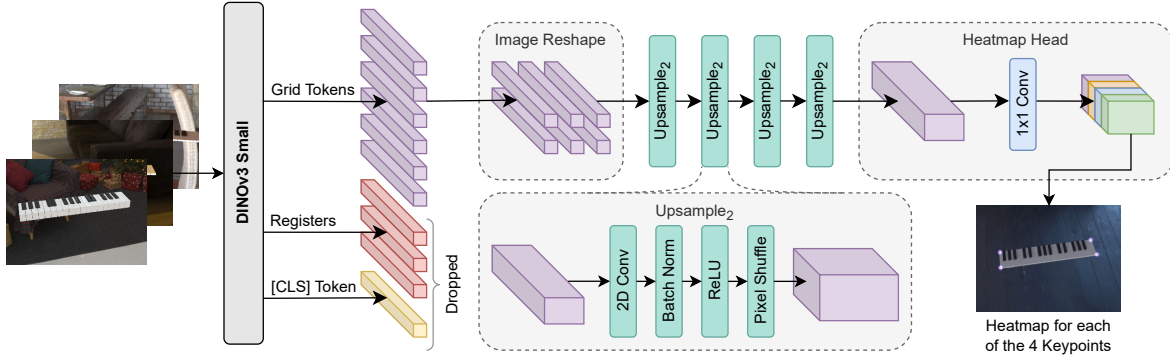


Figure 2. **Piano Detection Pipeline.** The input images are first fed to a small DINOv3 [25] feature extractor with fixed weights. These per-patch features are then forwarded to the shallow decoder head, which progressively refines and upsamples the intermediate features to a per-pixel representation. After mapping these features to per-keypoint heatmaps via a 1×1 convolution and sigmoid activation function, we can extract 2D keypoint coordinates by taking the maximum.

Realistic Image Synthesis For high-fidelity image generation, we utilize the physically-based renderer Mitsuba [12]. The scene is constructed by sampling high-resolution environment maps (HDRIs) for lighting and placing a randomly generated piano mesh at the origin. By applying realistic plastic BRDFs (Bidirectional Reflectance Distribution Functions) to the materials, we simulate the specular highlights and reflections typical of piano surfaces. Furthermore, as the exact 3D vertex coordinates are known, we can precisely project the target keypoints into image space, providing perfect ground-truth labels for the machine learning pipeline.

3.1.2 Keypoint Detector

The piano detection task can formally be described as follows: given an RGB input image $I \in \mathbb{R}^{H \times W \times 3}$, we aim to learn a mapping that returns a set of 2D keypoints $\mathbf{p} \in \mathbb{R}^{K \times 2}$. Inspired by the literature on human pose estimation [2, 31], we use a heatmap-based approach, where instead of directly regressing 2D keypoint locations, we predict 2D heatmaps $\mathbf{h} \in \mathbb{R}^{H \times W \times K}$ for the K keypoints. These heatmaps encode the model’s confidence that a given keypoint is present at each location in the input image. The 2D coordinates can then be recovered by taking the maximum.

To learn the detector from data, we parameterize this mapping by a neural network $f_\theta : \mathbb{R}^{H \times W \times 3} \rightarrow \mathbb{R}^{H \times W \times K}$. We further decompose this network into an encoder $\mathcal{E}_\phi$ and a decoder $\mathcal{D}_\psi$, such that $f_\theta(I) = \mathcal{D}_\psi(\mathcal{E}_\phi(I))$. For the encoder, we chose a small, pre-trained DINOv3 [25] feature extractor due to its rich semantic feature space, learned from internet-scale data. This allows us to keep the encoder weights ϕ frozen and only train a lightweight CNN-

based decoder, $\mathcal{D}_\psi$, that maps patch-level features from DINOv3 to pixel-level heatmaps. The decoder consists of a sequence of convolutional upsample layers that progressively increase the resolution. More specifically, within an upsampling layer, we first expand the intermediate feature map along the channel dimension using a padded 3×3 convolution ($H \times W \times C \rightarrow H \times W \times 2C$). This is then rearranged into a shallower, but higher resolution feature map of shape $2H \times 2W \times C/2$ via a pixel shuffle operation [24]. Since DINOv3 uses a patch size of 16×16 , we need 4 such upsampling layers to arrive at the original input resolution. Finally, we run the per-pixel representation through a 1×1 convolutional layer followed by a sigmoid activation to obtain the per-keypoint heatmaps. A visual diagram of our pipeline is depicted in Fig. 2.

Our loss function consists of the L_2 distance between the predicted and the ground-truth heatmap:

$$\mathcal{L}(\theta) = \|f_\theta(I) - \hat{\mathbf{h}}\|_2^2 \quad (1)$$

Where $\hat{\mathbf{h}}$ is obtained by placing an isotropic Gaussian kernel at the 2D keypoint location $\mathbf{p} = (p_x, p_y)^\top$.

$$\hat{\mathbf{h}}(\mathbf{x}) = \exp - \frac{\|\mathbf{p} - \mathbf{x}\|_2^2}{2\sigma^2} \quad (2)$$

To improve training speed, we start with a large standard deviation, $\sigma = 6.0$, and linearly decay it to $\sigma = 1.0$ as training progresses. This avoids diminishing gradients during the early stage, where the prediction is far from the ground truth.

3.2. Bongo Detection

The ambiguity of the keypoints makes them highly unsuitable for our synthetic dataset approach from before. We

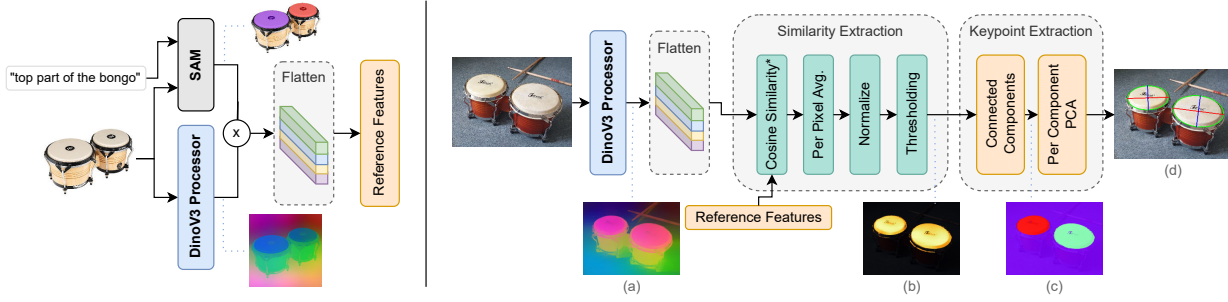


Figure 3. **Bongo Detection Pipeline.** To avoid training a detection model, we design a single-shot detection pipeline built on DINOv3 [25] features. More specifically, we extract a segmentation mask and features from a reference image in advance and store them for later use (on the left). At inference time, we compare each reference feature to all per-pixel features in the input image using cosine similarity. Thanks to DINOv3’s semantically meaningful latent space, visualized in (a), this simple operation yields high activations in the corresponding regions of the input image (the top two parts). This similarity map is then post-processed to produce a preliminary binary segmentation map (b). After extracting the two largest connected components (shown in (c) as red and cyan) and performing 2D principal component analysis, we obtain two ellipses in (d) that enclose the desired parts of the Bongos. Finally, we sample three boundary points for each ellipse and use them in our application.

therefore aim to design a Bongo detection pipeline that is training-free, efficient, and robust. While promptable general-purpose segmentation models like SAM3 [3] exist, their large scale prohibits efficient deployment on low-resource devices such as virtual reality headsets.

Our idea, therefore, is to leverage the semantic latent space of DINOv3, which can be computed on-device efficiently. We do this by first extracting the segmentation for a single fixed reference image of Bongos offline using SAM3 with the prompt: *top part of the bongo*. After bilinearly upscaling the patch-level features to the original resolution, we store the feature vectors $\mathbf{f}_{\text{ref}} \in \mathbb{R}^{N_{\text{ref}} \times D}$ for pixels within the segmentation mask for later use.

At inference time, we extract the DINOv3 features for the input image and upscale them to $\mathbf{f}_{\text{inp}} \in \mathbb{R}^{H \cdot W \times D}$. We then compute the dot product (unnormalized cosine similarity) between the segmented features of the reference image and all pixels of the input image.

$$S_{\text{ref} \rightarrow \text{inp}} = \mathbf{f}_{\text{ref}} \cdot \mathbf{f}_{\text{inp}}^{\top} \in \mathbb{R}^{N_{\text{ref}} \times H \cdot W} \quad (3)$$

We then aggregate the values to get per-pixel activations:

$$S_{\text{inp}} = \frac{1}{N_{\text{ref}}} \sum_{i=1}^{N_{\text{ref}}} (S_{\text{ref} \rightarrow \text{inp}})_{i,:} \in \mathbb{R}^{H \cdot W} \quad (4)$$

After normalizing and thresholding (with $\tau = 0.75$), we obtain a segmentation map $M \in \{0, 1\}^{H \cdot W}$ with 1-values in the top regions of the Bongos in the input image. To further refine our segmentation, we compute the two largest connected components of this activation map to remove spurious detections. These two components represent both top parts of the Bongos. We finally perform 2D principal

component analysis to convert each component’s segmentation into an ellipse. From the ellipse, we extract three deterministic boundary points to define the playable area, which is then used to display incoming musical cues. We refer the reader to Fig. 3 for a visualization of the pipeline’s intermediate outputs.

4. Unity Application

This section describes details about the Unity application and the integration of our models onto the headset.

4.1. Detection

To guide the user through the detection process, we designed intuitive visual tutorials for each method, as illustrated in Sec. 5. To improve system stability, we implement a heuristic constraint that assumes all detected keypoints for the piano, as well as those for each bongo, share a common y -coordinate and lie on the same xz -plane. To further enhance the user experience, we introduced *corrector nodes*, which allow the user to manually translate the 3D keypoints to compensate for sensor noise or minor misalignments.

4.1.1 Manual Detection

The manual detection of the keypoints described in Sec. 3 is done by shooting rays from the controllers to the piano, using the Environment Raycasting API from Meta [17]. Inferring from the camera’s depth-map stream, it can approximate an intersection point. Note that at the time, this feature was still in Beta, so API changes and many improvements are expected in the future. These 3D points are then parsed to the instrument geometries.

4.1.2 Automatic Detection

The automatic detection leverages our developed ML pipelines by running them on the Meta Quest 3, given the passthrough camera feed [18], with Unity’s AI inference engine [28]. Meta’s onboard implementation of the AI Inference Engine does not yet support all the operators used by the library. Therefore, our piano model can run on the GPU with an inference time of approximately 200ms, whereas the bongo detection uses GPU-incompatible operators and runs on the CPU, taking approximately 800ms per inference step. With a single button press, the user captures the camera feed, infers the keypoints using one of the models, and backprojects them to 3D space.

4.2. Visual Guidance

Piano and Bongos use processed MIDI data, providing each note’s start time t_n^{start} , duration t_n^{duration} , and key k_n .

4.2.1 Piano

A detected 2D plane is used as the target for falling blocks. Each block reaches the bottom of the plane at t_n^{start} . With a fixed fall time $t^{\text{fall}} = 2\text{ s}$, blocks spawn at $t_n^{\text{spawn}} = t_n^{\text{start}} - t^{\text{fall}}$. With constant velocity $v = h^{\text{plane}} / t^{\text{fall}}$, the block height is set by the note duration: $h_n = v \cdot t_n^{\text{duration}}$. Horizontal positions and widths are pre-computed for each piano key (including black and white keys) and selected using k_n .

4.2.2 Bongos

The same timing model is used on two circular surfaces: *hembra* (left) and *macho* (right). Semi-circular cues spawn at a fixed distance and move inward so they reach the drum edge at t_n^{start} . Notes are mapped only to left or right drum, each with a distinct color.

4.3. Design choices

To ensure intuitive interaction, we implemented visual cues that mirror the physical instruments: piano notes are represented by black and white blocks, while bongos use cyan and magenta color-coding for the *hembra* and *macho* (Fig. 1). To maintain a visual indication whether the registration is still accurate, we render the border of the detected instruments. This can be used as a real-time tracking monitor, allowing to detect drift or physical movement. A manual calibration fallback is available if environmental lighting degrades the automatic pipeline. Finally, mitigating the cost of switching from controllers to instruments, we added a 3-second countdown after all users confirmed readiness, providing enough temporal buffer to change.

4.4. Multiplayer

To facilitate cooperative learning and ensemble performance, the application incorporates a multiplayer framework built on the Photon engine [19]. This architectural choice enables multiple devices to synchronize musical sessions in real time, allowing users to engage in simultaneous performances ranging from duets to larger virtual orchestras. The implementation is designed to keep network traffic to a minimum with simple logic to synchronize and manage rooms of players. Because the multiplayer focuses on coordinating song playback, the system enables heterogeneous instrument collaboration.

5. Evaluation

5.1. Automatic Piano Detection

In this section, we ablate two variants of our Piano Keypoint Detector model from Sec. 3.1.2. One that uses transposed convolutions [32] for the upsampling layers, and one that uses regular convolutions and pixel shuffle operations [24]. The former was used during early experiments and was then discarded due to the typical block artifacts caused by transposed convolutions (see Fig. 4).

5.1.1 Training Setup

For our experiments, we create 100k samples at a resolution of 304×224 using our synthetic dataset pipeline from Sec. 3.1.1. We hold out 10% for validation and use the remaining samples for training. We also validate on 58 real images captured from our own piano to assess the synth-to-real gap.

We train both models for 50 epochs with a batch size of 128 using the Adam [14] optimizer with a fixed learning rate of 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, and without weight decay. The training completes in roughly 1 hour on a single NVIDIA RTX 5090.

For the ground-truth heatmaps, we linearly decay σ from 6.0 to 1.0 with respect to the current epoch.

The encoder is a frozen DINOv3 [25] backbone using the ViT-Small [6] architecture with a patch size of 16×16 for fast inference. The same backbone is also used for the Bongo detection pipeline (Sec. 3.2 and Fig. 3).

5.1.2 Ablation Study

We report performance across four commonly used metrics for 2D keypoint detection: the End Point Error (EPE), which is simply the Euclidean distance between the keypoints in pixel units; the Normalized Mean Error (NME), which is similar to EPE but divides the distance by the length of the bounding box diagonal that encloses the ground-truth keypoints d_{diag} ; the Percentage of Correct

Keypoints (PCK), which counts the fraction of keypoints that lie within a distance of $\alpha \cdot d_{\text{diag}}$ for different scales $\alpha \in \{0.05, 0.1, 0.15, 0.2\}$; and finally the PCK Area Under Curve (AUC) which approximates the integral of the PCK from 0 to $\alpha_{\text{max}} = 0.3$, normalized by α_{max} (we use 100 steps). Informally, the EPE and NME measure the overall accuracy of the model, while the PCK and AUC indicate how many keypoints fall within a satisfactory range usable for the downstream application.

For each dataset-model combination, we select the checkpoint corresponding to the epoch with the lowest ECE.

Our final version using PixelShuffle significantly outperforms the ConvTranspose variant, both quantitatively (Tab. 1) and qualitatively (Fig. 4). More specifically, we observe a roughly $2\times$ reduction of the distance metrics (EPE and NME), indicating that the PixelShuffle variant provides more accurate estimates. Moreover, we find that the synth-to-real gap is significantly smaller for the PixelShuffle model and more stable across different PCK scales. This shows that the model produces usable keypoints more frequently (up to almost 50%) and overfits less on synthetic data. We explain this effect by observing that the heatmaps produced by the PixelShuffle model are substantially smoother and of higher fidelity than those from the ConvTranspose model, which yields block artifacts that are more sensitive to small model errors. Fig. 4 visualizes the heatmap output for one keypoint on real data samples.

5.2. User Study

XR applications frequently struggle with the trade-off between feature richness and ease of onboarding. Complex calibration steps or text-heavy tutorials can deter users before they even begin playing. To evaluate our design choices and ensure our application avoids these common pitfalls, we conducted a [user study](#) (Sec. A) during the demo day.

In total, **19 participants** with varying backgrounds and levels of experience utilizing Mixed Reality headsets participated in the study.

5.2.1 Metrics and Methods

We gathered feedback through a combination of observations during the sessions and a post-experience questionnaire. The questionnaire consisted of both open-ended qualitative questions to identify specific pain points and quantitative metrics, such as Likert scales, to validate our architectural decisions.

5.2.2 Qualitative Results

The open-ended responses revealed that the application’s core workflow was generally clear, with most participants



Figure 4. **Qualitative Comparison.** Side-by-side comparison of heatmap predictions on real piano images. PixelShuffle (left) produces more accurate and robust corner localizations than ConvTranspose (right) across diverse viewing angles and lighting conditions. This figure also illustrates the block artifacts of transposed convolutions.

reporting no issues during navigation. The feedback collected primarily focused on specific usability enhancements and feature requests rather than fundamental flaws in the interaction design.

Calibration Handles Although quantitative data shows that the manual calibration phase was well received (see Sec. 5.2.3), users specifically noted that the “white cubes” used for resizing and moving the instrument surface were confusing to interact with and lacked clear affordances regarding their direction of movement, hence we reiterated that feature in Sec. 5.3.

5.2.3 Quantitative Results

We analyze the quantitative responses and preference choices to validate our UX approach and the technical implementation of the instrument detection.

UI & Visuals The decision to use a modern, icon-based UI was strongly validated. 62.5% of users rated the visuals a 5/5, and 68.8% explicitly preferred the “Modern Icon-

Dataset	Model	Epoch	EPE (px) ↓	NME (10^{-2}) ↓	PCK@0.05 ↑	PCK@0.10 ↑	PCK@0.15 ↑	PCK@0.20 ↑	AUC ↑
Synthetic	ConvTranspose	30	2.39	1.16	0.987	0.998	0.999	1.000	0.961
	PixelShuffle	20	1.34	0.60	0.991	0.999	1.000	1.000	0.981
Real	ConvTranspose	30	25.43	12.44	0.323	0.457	0.625	0.772	0.596
	PixelShuffle	10	11.74	5.64	0.776	0.819	0.866	0.914	0.819

Table 1. **Quantitative Ablation Study.** We observe that PixelShuffle predicts keypoints roughly $2\times$ more accurately than ConvTranspose on both the synthetic (1.34px vs. 2.39px) and real (11.74px vs. 25.43px) datasets. This is likely due to the block artifacts typical of transposed convolutions, which cause discontinuities and jumps in the prediction. Moreover, PixelShuffle significantly outperforms ConvTranspose on PCK (almost 50% more keypoints land within a usable range at the smallest scale), indicating fewer outliers on average.

Based” design over a text-based alternative. When asked if they would prefer a “traditional UI,” the majority rejected the idea (31.3% rated the desire for a traditional UI as 1/5, and 25% as 2/5), confirming our design direction.

Intuitiveness The app was generally considered intuitive, with 73.7% of users rating it a 4 or 5 out of 5. Only a small minority (5.3%) reported difficulties distinguishing colors, suggesting our accessibility choices were successful.

Detection Methods: Automatic vs. Manual One of the core technical contributions of this project is the Automatic Instrument Detection. The user study reveals that the perceived utility of this automation is closely tied to the geometric complexity of the instrument. While the automatic pipeline was the clear preference for the intricate structure of the piano, the simpler configuration of the bongos resulted in more balanced satisfaction between methods:

- **Piano:** The automatic detection was the clear winner. 66.7% of users explicitly preferred the Automatic method. In terms of understandability, 94.1% rated the Automatic UI positively (3 or higher), compared to 78.6% for the Manual UI.
- **Bongos:** Results for the bongos ($N=10$) showed a higher acceptance of the manual process compared to the piano, likely due to the simpler geometry. While 50% of users rated the Automatic Bongo UI a perfect 4/4 (compared to 30% for Manual), the overall preference was split, with users appreciating the control offered by the manual fallback.

Overall Experience The overall enjoyment was high, with 76.5% of users rating the piano-playing experience 4/4 (Very Good). Most importantly, regarding the “Fun” factor, 84.6% of participants rated the app a 4 or 5 out of 5.

Statistical Analysis Due to the limited sample size ($N = 19$), Mann-Whitney U tests for all primary metrics showed no statistically significant effects ($p > 0.05$). However,

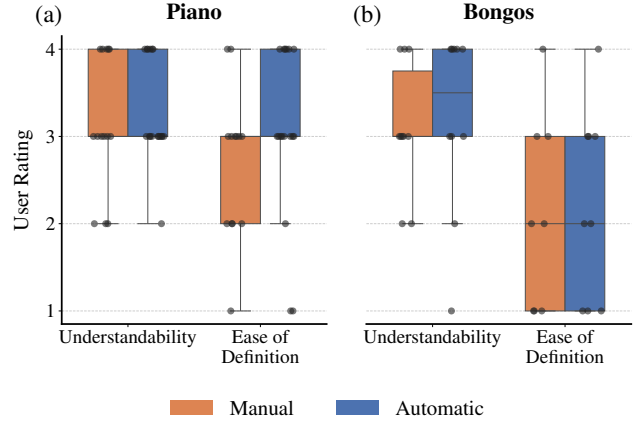


Figure 5. **Evaluation of Instrument Detection Methods.** User ratings comparing Automatic (blue) and Manual (orange) calibration across two dimensions: UI Understandability and Ease of Definition. Responses are plotted on a 5-point Likert scale (1 = Poor/Very Hard, 5 = Excellent/Very Easy), with individual data points overlaid ($N = 19$).

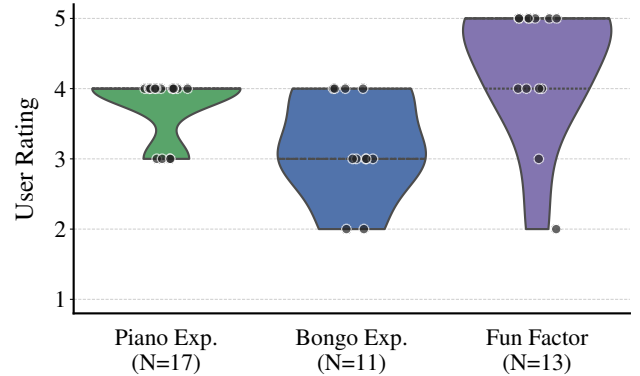


Figure 6. **User Experience and Enjoyment.** Participants rated their playing experience for the Piano and Bongos on a 4-point scale, and the overall “Fun Factor” of the application on a 5-point scale. The Piano experience was rated highly positive (Median=4), while the Bongos showed more variance.

as shown in Tab. 2, the automatic method consistently

achieved higher descriptive means and was preferred by 66.7% of participants. Analyses for the bongos were omitted due to a sample size of $N = 8$.

Metric	Automatic (M)	Manual (M)	p -value
Ease of Definition	3.29	3.00	> 0.05
UI Understandability	3.29	3.14	0.31
Overall Preference	66.7%	33.3%	0.11

Table 2. Piano Statistics ($N = 19$).

5.3. Design Implications and Iterations

Based on the direct feedback from the study, we implemented several key changes to improve the final user experience.

Replacing Text with Visual Instructions While some users asked for more text labels in the qualitative feedback, our observation during the demo day was that users tended to ignore large blocks of instructional text in VR. To address the confusion regarding the user flow without reintroducing cognitive overload, we removed large text instructions. Instead, we replaced them with intuitive visual diagrams (see Fig. 7) that show the required controller inputs.

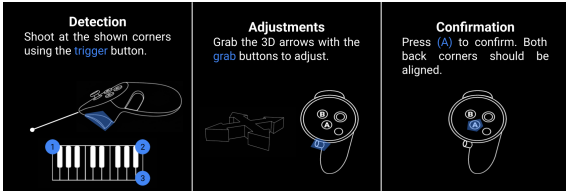


Figure 7. Visually guided instructions instead of text-heavy.

Visual Feedback and Note Timing Users noted that falling blocks could blend into the background or each other. To address this, we implemented dynamic visual feedback. Notes now change color (*e.g.*, turning blue) the moment they reach the “hit” zone (see Fig. 1). This provides immediate confirmation to the user and helps distinguish the timing of consecutive notes, directly addressing the visibility concerns raised in the study.

Calibration UI Overhaul We replaced the white corrector cubes with intuitive 3D arrows. These arrows clearly indicate the axes of translation, making the manual adjustment of the piano surface significantly more intuitive for users who need to fall back to manual detection.

6. Discussion & Future Work

Our system makes three significant contributions: (1) accurate on-device instrument detection (≈ 200 ms for piano) using efficient frozen DINOv3-Small encoders with lightweight decoder architectures, eliminating external dependencies; (2) the SynthesiaSet, a synthetic piano dataset with precise geometric annotations from AR/VR perspectives, where our parametric mesh generator captures dimensional variability across manufacturers; and (3) a training-free bongo detection pipeline leveraging single-shot reference features in DINOv3’s latent space, demonstrating efficient design for variable-geometry instruments without costly dataset collection.

The most significant limitation is the requirement for pre-gameplay instrument registration with a locked spatial configuration. Running robust keypoint detection concurrently with high-fidelity rendering imposes substantial computational overhead—the bongo pipeline’s CPU-bound execution (≈ 800 ms) reveals current hardware constraints, as the Meta Quest 3’s AI Inference Engine lacks full operator support. While this maintains the 72Hz refresh rate necessary for comfortable VR, it limits robustness to drift and prevents repositioning during practice.

As our current bongo model requires specific operations that cannot be run on the Meta Quest GPU at the time of writing, a different model could be designed to circumvent this issue. Further, the current application only supports 3 songs, limiting the range of musical versatility. Future work could add a connection to a MIDI-hosting server and directly parsing songs on the fly. Whereas MIDI-to-piano mapping is defined by convention, our bongo translation builds on intuitive heuristics. Thus, additional song files could be tested.

Future work could build on the baseline we provide by adding more instruments to our current catalog. Though a priority would be dynamic instrument tracking via continuous re-registration using optical flow or lightweight pose-refinement networks. Integrating audio analysis for real-time performance assessment would enable adaptive difficulty and personalized learning.

7. Conclusion

We present a standalone mixed reality system for music learning that performs automatic instrument detection and spatial registration directly on the Meta Quest 3. By combining on-device inference, synthetic data generation, and dedicated detection pipelines for piano and bongos, our approach supports multiple instruments without requiring manual calibration and external dependencies. Our results demonstrate that accurate, low-latency instrument registration is feasible on consumer hardware, laying the groundwork for scalable, extensible mixed reality music tutors.

References

- [1] Nicolò F Bernardi, Mohammad Darainy, and David J Ostry. Somatosensory contribution to the initial stages of human motor learning. *Journal of neuroscience*, 35(42):14316–14326, 2015. 2
- [2] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7291–7299, 2017. 3
- [3] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. 2, 4
- [4] Jonatan Crafoord. Virtuoso, 2022. 2
- [5] Michael Dawson-Haggerty et al. trimesh, 2025. 2
- [6] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 5
- [7] Michael Garland and Paul S Heckbert. Surface simplification using quadric error metrics. In *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*, pages 209–216, 1997. 2
- [8] Mark Gurman. A first look at the headset that could be Apple’s biggest competition. 2
- [9] Dominik Hackl. Magic keys:: Mixed reality piano learning, Sep 2022. 2
- [10] Dominik Hackl and Christoph Anthes. Holokeys-an augmented reality application for learning the piano. In *Forum media technology*, pages 140–144, 2017. 2
- [11] David Heaney. Quest 3 specs, compared to quest 2 & apple vision pro. 1, 2
- [12] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. Dr.jit: A just-in-time compiler for differentiable rendering. *Transactions on Graphics (Proceedings of SIGGRAPH)*, 41(4), July 2022. 3
- [13] Ismail Mohamed Jamal and Erdal Kiliç. Easyarpiano: Piano teaching mobile app with augmented reality. In *2021 International Conference on Forthcoming Networks and Sustainability in AIoT Era (FoNeS-AIoT)*, pages 66–71, 2021. 2
- [14] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 5
- [15] Charles Loop. Smooth subdivision surfaces based on triangles. *Master’s thesis, University of Utah, Department of Mathematics*, 1987. 2
- [16] Pavel Marceluch. Vrtuos, Nov 2020. 2
- [17] Meta Platforms, Inc. Environment raycast (mruk), 2025. 2, 4
- [18] Meta Platforms, Inc. Passthrough camera api overview, 2025. 2, 5
- [19] Photon Engine. Photon Unity Networking (PUN) — Multiplayer for Unity, 2026. 5
- [20] Zac Reid. Pianovision - home, Sep 2023. 2
- [21] Katja Rogers, Amrei Röhligh, Matthias Weing, Jan Gugenheimer, Bastian Könings, Melina Klepsch, Florian Schaub, Enrico Rukzio, Tina Seufert, and Michael Weber. Piano: Faster piano learning with interactive projection. In *Proceedings of the Ninth ACM International Conference on Interactive Tabletops and Surfaces*, pages 149–158, 2014. 2
- [22] Naotaka Sakai. Keyboard span in old musical instruments: Concerning hand span and overuse problems in pianists. *Medical Problems of Performing Artists*, 23(4):169–171, 2008. 2
- [23] PotamWorks SAS. Smash drums!, 2020. 2
- [24] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1874–1883, 2016. 3, 5
- [25] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025. 3, 4, 5
- [26] John Sweller and Paul Chandler. Why some material is difficult to learn. *Cognition and Instruction*, 12(3):185–233, 1994. 1
- [27] Emre Tanirgan. Play drums without the limitations of the real world, Mar 2024. 2
- [28] Unity Technologies. *Unity Sentis User Manual (Version 2.2)*. Unity Documentation, 2025. Package: com.unity.ai.inference. 5
- [29] Jörg Vollmer, Robert Mencl, and Heinrich Mueller. Improved laplacian smoothing of noisy surface meshes. In *Computer graphics forum*, volume 18, pages 131–138. Wiley Online Library, 1999. 2
- [30] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information, 2024. 2
- [31] Yufei Xu, Jing Zhang, Qiming Zhang, and Dacheng Tao. Vitpose: Simple vision transformer baselines for human pose estimation. *Advances in neural information processing systems*, 35:38571–38584, 2022. 3
- [32] Matthew D Zeiler, Dilip Krishnan, Graham W Taylor, and Rob Fergus. Deconvolutional networks. In *2010 IEEE Computer Society Conference on computer vision and pattern recognition*, pages 2528–2535. IEEE, 2010. 5

A. User Study Questionnaire

AR Musician User Study - GUI

263-5905-00L - Mixed Reality User Study

** Indicates required question*

1. How much did you like the visuals of our UI? ***

We tried to keep the UI as intuitive as possible by disabling buttons which should not be clickable at some state. To improve HCI, the "single player" button has been specifically placed on the left side, as this is the most important button.



Mark only one oval.

12345

Ugly
☐
☐
☐
☐
☐
Very Nice

2. How intuitive was the UI? ***

Mark only one oval.

12345

Not
☐
☐
☐
☐
☐
Very Intuitive

3. Was there something unclear while navigating through the app?

4. How much would you like the visuals of a more boring, but traditional UI? ***



Mark only one oval.

12345

Ugly
☐
☐
☐
☐
☐
Very Nice

5. Which UI type would you prefer? ***

Mark only one oval.

1234

Mod
☐
☐
☐
☐
Text-Based

6. Did you have problems distinguishing certain colors? ***

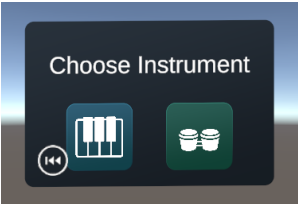
Mark only one oval.

☐ Yes

☐ No

7. If you did have any problem distinguishing certain colors, which ones?

8. How necessary do you think it is to include "Titles" in the intermediate steps to choose an instrument for example? *



Mark only one oval.

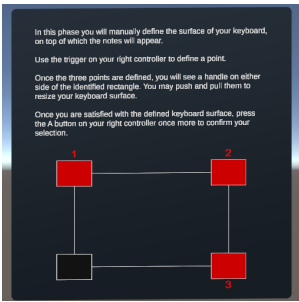
1 2 3 4

Not ☐ ☐ ☐ ☐ Very Necessary

9. If any instruction was unclear, which one and why?

AR Musician User Study - Piano

10. How understandable is the Manual Piano Detection UI? Only select an answer if you have tried it.

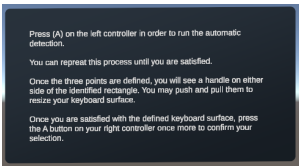


Mark only one oval.

1 2 3 4

Not ☐ ☐ ☐ ☐ Very Understandable

11. How understandable is the Automatic Piano Detection UI? Only select an answer if you have tried it.

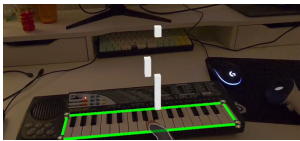


Mark only one oval.

1 2 3 4

Not ☐ ☐ ☐ ☐ Very Understandable

12. How much do you like the visuals of the Piano?



Mark only one oval.

1 2 3 4

Ugly ☐ ☐ ☐ ☐ Very Nice

13. How easy was it to define the piano overlay?

Mark only one oval per row.

	Very Easy	Easy	Hard	Very Hard	Didn't try
Automatic	☐	☐	☐	☐	☐
Manual	☐	☐	☐	☐	☐

14. How good was the alignment of the piano?

Mark only one oval.

1 2 3 4

Unaligned ☐ ☐ ☐ ☐ Very Good

15. Which piano detection method do you prefer?

Mark only one oval.

1 2 3 4

Manual ☐ ☐ ☐ ☐ Automatic

16. Overall, how was your piano playing experience?

Mark only one oval.

1 2 3 4

Very ☐ ☐ ☐ ☐ Very Good

17. What would you change on the piano?

AR-Musican User Study - Bongos

18. How understandable is the Manual Bongo Detection UI?
Only select an answer if you have tried it.

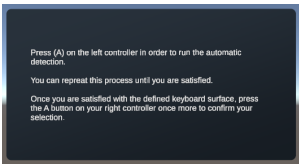


Mark only one oval.

1 2 3 4

Not ☐ ☐ ☐ ☐ Very Understandable

19. How understandable is the Automatic Bongo Detection UI?
Only select an answer if you have tried it.



Mark only one oval.

1 2 3 4

Not ☐ ☐ ☐ ☐ Very Understandable

20. How much do you like the visuals of the Bongos?



Mark only one oval.

1 2 3 4

Ugly ☐ ☐ ☐ ☐ Very Nice

21. How easy was it to define the bongo overlay?

Mark only one oval per row.

	Very Easy	Easy	Hard	Very Hard	Didn't try
Automatic	☐	☐	☐	☐	☐
Manual	☐	☐	☐	☐	☐

22. How good was the alignment of the bongos?

Mark only one oval.

1 2 3 4

Unal ☐ ☐ ☐ ☐ Very Good

23. Which bongo detection method do you prefer?

Mark only one oval.

1 2 3 4

Man ☐ ☐ ☐ ☐ Automatic

24. Overall, how was your bongo playing experience?

Mark only one oval.

1 2 3 4

Very ☐ ☐ ☐ ☐ Very Good

25. What would you change on the bongos?

26. Most importantly, how much fun did you have trying out our app?

Mark only one oval.

1 2 3 4 5

Suff ☐ ☐ ☐ ☐ ☐ Very Enjoyable